

Examen réparti 2

12 janvier 2021

Antoine Miné

Déroulement de l'examen

L'examen dure 2h.

Tous les documents papier sont autorisés (impressions des transparents, notes personnelles, livres, etc.). L'utilisation de moyens de communication (téléphone, tablette, etc.) est interdite. L'utilisation d'une clé USB est autorisée, en lecture seule. Seul le format FAT est supporté (pas NTFS).

Vous trouverez sur votre compte dans le répertoire de travail `eclipse-workspace` un projet Eclipse `ILP4` contenant les sources d'ILP4. Vous complétez ce projet en répondant aux questions de l'énoncé. Assurez-vous de placer les fichiers demandés dans le package (et donc dans le répertoire) précisé pour chaque question. En fin d'examen, sauvegardez tous vos fichiers, fermez Eclipse et déloguez-vous. Les fichiers de votre compte seront automatiquement ramassés.

Le point de départ de l'examen est la version **ILP4**. Tous vos fichiers doivent être rédigés dans le package `com.paracamplus.ilp4.exam2020` et ses sous-packages du projet Eclipse `ILP4`. Ils se trouveront donc dans un sous-répertoire de `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2020` de votre compte.

L'examen sera corrigé à la main. Quand du code est demandé, les erreurs de syntaxe ne seront pas pénalisées. Il est souhaitable de n'explicitier que les parties du code qui vous semblent importantes, et de remplacer les parties peu informatives par un commentaire indiquant succinctement ce qui a été omis (inutile, par exemple, le détailler du code répétitif dans son intégralité).

Mise en route

- Ouvrez un navigateur et allez à l'URL : `http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP`. Vous y trouverez notamment les transparents du cours. Une copie locale de la documentation Java est disponible sur `https://www-ppti.ufr-info-p6.jussieu.fr`.
- Lancez Eclipse en tapant dans un terminal : `eclipse-2020 &`
- Ouvrez le projet `ILP4` contenant les sources d'ILP4 et vérifiez qu'il compile bien.
- Si le projet `ILP4` n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `eclipse-workspace/ILP4`, puis « Finish ».
- Le plug-in ANTLR ne fonctionne pas toujours. Pour compiler les grammaires ANTLR 4 en sources Java, vous devez taper dans un terminal :

```
cd ~/eclipse-workspace/ILP4
./compile_ANTLR.sh
```

puis, dans Eclipse, cliquez sur le projet dans le « Package explorer » et tapez F5 (ou « Refresh »). Les sources générées par ANTLR 4 devraient apparaître dans `targets/generated-sources/antlr4`. Il faut attendre quelques secondes avant qu'elles soient compilées.

Notez que, après chaque ajout ou modification d'un fichier `.g4`, vous devrez à nouveau relancer la commande `./compileANTLR.sh` depuis le répertoire `eclipse-workspace/ILP4`.

- Une cause possible d'erreur de compilation est un environnement Java mal configuré. Faites un clic droit sur le nom du projet dans le « Package Explorer », cliquez sur « Properties », puis :
 - Dans l'onglet « Java build path / Libraries », vérifiez la version de la bibliothèque système utilisée (« JRE System Library »). Vous pouvez la changer avec le bouton « Edit... / Alternate JRE » et choisir un JDK approprié (JDK 10 ou supérieur). Les JDK disponibles localement sont dans le répertoire `/usr/lib/jvm`
 - Vérifiez que JUnit 4 est présent. Sinon, ajoutez-le avec le bouton « Add Library ».
 - Vérifiez la version de compatibilité Java dans l'onglet « Java Compiler » des propriétés. Il doit correspondre à la version du JDK utilisée.
 - Vérifiez que la liste des répertoires source contient le répertoire `target/generated-sources/antlr4` (dans « Java build path », onglet « Source »).

Introduction

Dans cet examen nous allons ajouter à ILP4 un support partiel pour les **types sommes**, aussi appelées unions discriminées ou variants. Une valeur d'un type somme est composée d'un discriminant (appelé aussi *tag*) et de zéro, un ou plusieurs arguments. Nous construisons une nouvelle valeur somme avec le symbole `$`. Par exemple :

```
1    x = $A(1,2,3);
2    y = $Test();
3    z = $Etudiant("John","Smith",2+2);
```

à la ligne 1, nous stockons dans `x` une valeur somme de discriminant « A » ayant trois arguments : 1, 2 et 3 ; à la ligne 2, dans `y`, une valeur somme de discriminant « Test » sans argument ; et à la ligne 3, dans `z`, une valeur somme de discriminant « Etudiant ». Le symbole `$` est utilisé pour éviter de confondre la construction d'une valeur somme avec un appel de fonction. Le discriminant, le nombre et la valeur des arguments sont libres.

Une valeur somme peut être inspectée avec la construction `match-with` suivante, aussi appelée filtrage :

```
1    res1 = match x with A(a,b,c) then (a+b+c) else -1;
2    res2 = match y with A(a,b,c) then (a+b+c) else -1;
```

qui stockera, à la ligne 1, 6 dans `res1` car `x` est bien de type somme de discriminant A, donc l'expression `a+b+c` de la branche `then` est évaluée dans un contexte où `a`, `b` et `c` sont liés aux arguments de la somme (c'est à dire 1, 2 et 3) et retourne bien $1 + 2 + 3 = 6$. Comme `y` n'a pas le discriminant « A » attendu, la valeur `-1` est stockée dans `res2` à la ligne 2 (la branche `else` est évaluée).

Types sommes

Nous allons ajouter à ILP4 :

- un **nouveau type de valeurs ILP** correspondant aux types sommes ;
- l'opération de **construction**, permettant de créer une valeur somme ;
- l'opération de **filtrage**, permettant d'inspecter une valeur somme.

Valeurs de type somme. Une valeur somme pourra être représentée en mémoire par :

- le discriminant (une chaîne de caractères) ;
- un tableau de taille arbitraire de valeurs ILP.

Construction. La syntaxe générale de la construction est une expression de la forme suivante :

$$\text{\$tag}(\text{expr1}, \dots \text{exprN})$$

c'est à dire : le symbole dollar `$`, suivi du discriminant *tag*, suivi d'une liste entre parenthèses d'arguments *expr1* à *exprN* de taille arbitraire. Le discriminant peut être n'importe quelle séquence de lettres et de chiffres autorisée pour les noms des variables (i.e., un *identifieur*). Les arguments peuvent être des expressions arbitraires.

L'évaluation de l'expression `$tag(expr1, ... exprN)` aura pour effet d'évaluer tous les arguments, de construire une valeur somme ayant le discriminant et les valeurs spécifiées, et de retourner cette valeur somme.

Filtrage. La syntaxe générale du filtrage est une expression de la forme suivante :

```
match expr with tag(var1, ..., varN) then expr1 else expr2
```

qui comporte, outre les mot-clés `match`, `with`, `then` et `else` : une expression *expr*, un discriminant *tag*, une liste de variables *var1* à *varN* de taille arbitraire entre parenthèses, une expression *expr1* dénotant le corps de la branche *then* et une expression *expr2* dénotant le corps de la branche *else*.

L'évaluation de cette expression de filtrage commencera par évaluer *expr*, puis :

1. Si la valeur de *expr* est bien de type somme avec discriminant *tag*, alors la branche *then* est exécutée. La valeur retournée est la valeur de l'expression *expr1* évaluée dans un environnement lexical où les variables *var1* à *varN* sont liées aux arguments stockés dans la valeur somme.
2. Si la valeur de *expr* n'est pas de type somme, ou bien n'a pas le discriminant *tag*, alors la branche *else* est exécutée. La valeur retournée est la valeur de l'expression *expr2*. Cette expression est évaluée dans un environnement lexical inchangé, sans ajouter les variables *var1* à *varN*.

Notez que les variables *var1* à *varN* sont **locales à *expr1***.

Du fait de l'absence de typage statique en ILP, il n'est pas nécessaire de déclarer à l'avance les discriminants autorisés, ni de spécifier le nombre d'arguments attendus pour chaque discriminant. Il est possible que, quand la branche *then* du `match-with` est exécutée, le nombre de variables *var1* à *varN* diffère du nombre d'arguments stockés dans la valeur somme inspectée. Nous considérons que :

- ce n'est pas une erreur s'il y a plus d'arguments que de variables ; les arguments excédentaires ne sont pas utilisés ; ainsi le programme suivant est correct :

```
match $A("a","b","c") with A(x) then print(x) else false
```

- s'il y a strictement plus de variables que d'arguments, c'est une erreur qui sera signalée par une **exception ILP** (pour éviter de laisser des variables non définies) ; le programme suivant signale donc une erreur :

```
match $A(1) with A(x,y) then (x+y) else false
```

Exemple. Voici un exemple plus complet qui utilise les types sommes pour encoder des listes chaînées (similaires aux listes OCaml). Une liste est soit la liste vide, indiquée par la somme `Nil()` sans argument, soit un maillon `Cons(head,tail)` contenant l'élément en tête de liste dans `head` et une référence au reste de la liste dans `tail`. La fonction `length` suivante calcule récursivement la taille d'une liste, et indique une erreur si la structure contient des valeurs qui ne sont ni `Nil` ni `Cons` :

```
1 function length(a) (  
2   match a with Cons(x,y) then (length(y) + 1) else  
3   match a with Nil() then 0 else (throw "Not a list!")  
4 )  
5  
6 let list = $Cons("un", $Cons("deux", $Cons("trois", $Nil()))) in  
7 print(length(list))
```

L'exécution de ce programme devra afficher à l'écran 3 (la longueur de la liste `list`).

1 Grammaire et analyse syntaxique

Grammaire (2.5 pts). Écrivez une grammaire qui étend ILP4 avec la construction des valeurs somme `$tag(expr1,...,exprN)` et leur filtrage `match expr with tag(var1,...,varN) then expr1 else expr2`.

Cette grammaire se trouvera dans le fichier `eclipse-workspace/ILP4/Java/src/com/paracampus/ilp4/exam2020/grammar/ILPgrammarExam.g4`. N'oubliez pas de lancer `./compile_ANTLR.sh` pour compiler la grammaire en Java.

Arbre syntaxique (2.5 pts). Proposez une extension de l'arbre syntaxique pour gérer la construction et le filtrage des types sommes. Vous fournirez toutes les interfaces correspondantes dans le package `com.paracampus.ilp4.exam2020.interfaces` et toutes les classes dans le package `com.paracampus.ilp4.exam2020.ast` sans oublier celles liées aux motifs fabrique (*factory*) et visiteur (*visitor*).

Analyse syntaxique (2.5 pts). Implantez dans le package `com.paracampus.ilp4.exam2020.parser` les classes `ILPMLparser` et `ILPMLlistener` implantant l'analyseur syntaxique pour le langage étendu.

2 Interprétation

Structure de données (1.5 pt). Proposez dans `com.paracampus.ilp4.exam2020.interpreter` une classe `Sum` qui implante la représentation en mémoire d'une valeur somme lors de l'interprétation, ainsi que l'interface `ISum` correspondante dans `com.paracampus.ilp4.exam2020.interpreter.interfaces`. Une valeur somme pourra être représentée à l'aide de deux attributs : une chaîne qui représente le discriminant et un tableau de valeurs qui représente les arguments.

Interprète (4 pts). Implantez dans `com.paracampus.ilp4.exam2020.interpreter` la classe `Interpreter` qui enrichit l'interprète d'ILP4 avec la gestion des sommes, leur construction et leur filtrage. Vous prendrez garde à bien gérer tous les cas demandés (branche *then*, branche *else*, cas d'erreur).

3 Tests (2 pts)

Proposez une base de tests dans le répertoire `eclipse-workspace/ILP4/Java/src/com/paracampus/ilp4/exam2020/samples/`. Pour chaque test, fournissez le source ILPML mais aussi les fichiers permettant de vérifier les résultats attendus.

Vous prendrez garde à fournir suffisamment de tests pour couvrir l'ensemble des comportements possibles (y compris les cas d'erreur).

Vous fournirez un fichier texte `eclipse-workspace/ILP4/Java/src/com/paracampus/ilp4/exam2020/samples/tests.txt` précisant, pour chaque test, le comportement ou l'erreur testée.

4 Compilation

Classes et interfaces (1.5 pt). Donnez la liste des classes et interfaces que vous auriez à ajouter pour implanter le compilateur, en indiquant leur nom, leur package, les interfaces

qu'elles implantent et les classes et interfaces dont elles héritent. On ne demande pas dans cette question le code Java de ces classes et interfaces.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2020/compiler/classes.txt`

Analyse statique (2 pts). Donnez dans les classes `FreeVariableCollector` et `GlobalVariableCollector` du package `com.paracamplus.ilp4.exam2020.compiler` l'analyse statique des variables libres et des variables globales. On ne demande pas ces classes intégralement, mais seulement les méthodes qui gèrent le nœud filtrage.

Bibliothèque d'exécution (2 pts). Indiquez les modifications à apporter à la bibliothèque d'exécution pour supporter les types sommes. Si le type `ILP_Object` doit être enrichi, donnez le code C du nouveau type. Listez également les fonctions C que vous auriez à implanter, en précisant leur rôle et leur fonctionnement.

Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2020/C/ilp.txt`. Le code C complet de `ilp.c` et `ilp.h` n'est pas demandé.

Schéma de compilation (3 pts). Donnez un schéma de compilation détaillé pour la construction et le filtrage des sommes. Vous fournirez votre réponse dans un fichier texte : `eclipse-workspace/ILP4/Java/src/com/paracamplus/ilp4/exam2020/compiler/schema.txt`. Le code de la classe `Compiler` n'est pas demandé.

Précisez également dans le même fichier texte si les erreurs provoquées dans la gestion des types sommes peuvent être traitées statiquement ou dynamiquement.